

TrueNAS SCALE on Proxmox

HBA and GPU passthrough from BIOS to a running pool

Eric

2026-09-11

Table of contents

Preface	1
How to build	1
Scope	1
1 Overview	3
1.1 Target architecture	3
1.2 Why pass the HBA, not individual disks	3
1.3 Hardware checklist	4
2 BIOS and UEFI settings	5
2.1 Settings that matter	5
2.2 Where vendors hide IOMMU	7
2.3 Slot choice	7
3 Installing Proxmox VE	9
3.1 Preparing installation media	9
3.2 Booting the installer	9
3.3 Running the installer	10
4 Enable IOMMU on the Proxmox host	11
4.1 GRUB	11
4.2 systemd-boot	11
4.3 Load VFIO at boot	12
5 Identify the HBA and IOMMU group	13
5.1 Find the card	13
5.2 Print IOMMU groups	13
5.3 When the group is dirty	13
6 Bind the HBA to vfio-pci	15
6.1 Confirm IDs	15
6.2 Bind and blacklist	15
6.3 If the host driver still wins	16

7	Create the TrueNAS SCALE VM	17
7.1	Download the TrueNAS SCALE ISO	17
7.2	Settings that matter	18
7.3	Example <code>qm create</code>	18
8	Pass the HBA into the VM	21
8.1	From the Proxmox UI	21
8.2	From the CLI	21
8.3	After start	21
9	GPU passthrough	23
9.1	Identify the GPU and its group	23
9.2	Free up a console for the host	23
9.3	Bind the GPU to <code>vfiopci</code>	24
9.4	Pass the GPU into the VM	24
9.5	Known quirks	25
9.6	Verify	25
10	TrueNAS after passthrough	27
10.1	First checks	27
10.2	Guest hygiene	27
10.3	What not to do	27
11	Troubleshooting	29
11.1	VM hangs at “Booting from Hard Disk”	29
11.2	IOMMU groups missing / “PCI device not available”	29
11.3	Proxmox still lists the data disks	29
11.4	TrueNAS sees the HBA but no disks	29
11.5	Host dies when the VM autostarts	30
11.6	Need ACS override?	30
11.7	Guest shows a generic display adapter, not the GPU	30
11.8	Windows guest shows Nvidia Code 43	30
11.9	GPU passthrough works once, then fails after a VM reboot	30
11.10	Quick verification set	30
12	Command reference	31
12.1	Host	31
12.2	Kernel flags	31
12.3	<code>vfiopci</code>	32
12.4	VM	32
12.5	GPU passthrough (Chapter 9)	32

Preface

This book is a working runbook for running **TrueNAS SCALE as a VM on a Proxmox VE host**, with the data disks attached through a dedicated **HBA in IT mode** passed through to the guest. It also covers passing a **discrete GPU** into a second VM on the same host, since the BIOS and IOMMU groundwork is shared between both.

That is the layout iXsystems still recommends for production data: the hypervisor owns its own boot disk; TrueNAS owns the storage controller and the pool.

Use it as:

- a **Quarto book** (numbered chapters, cross-references, PDF/EPUB) — download the PDF
- a **website** (`quarto render → _book/index.html`, then publish with `quarto publish`)

How to build

In RStudio: **File → Open Project** on this folder, then **Render Book**.

From a terminal in this directory:

```
quarto preview
quarto render
quarto publish gh-pages
```

Scope

Chapters walk the host from a cold BIOS through installing Proxmox VE, IOMMU, vfio binding, the TrueNAS VM, the PCI passthrough line for the HBA, a parallel GPU passthrough setup, and the failures that usually show up after a device is attached.

Commands assume the current stable releases as of this writing: **Proxmox VE 9** (9.2, on Debian 13 “Trixie”) and **TrueNAS SCALE 25.10 “Goldeye”**.

Proxmox VE 8 (Debian 12 “Bookworm”) reached end of life in August 2026 — upgrade if you’re still on it. TrueNAS 26 exists as a beta line but isn’t the recommended stable release yet; this book targets 25.10. PCI addresses and vendor:device IDs in examples are placeholders. Replace them with output from your own `lspci`.

💡 Check which Proxmox VE you’re on

```
pveversion -v
```

The first line’s `proxmox-ve:` field starts with `9.` on a current install or `8.` on the now-EOL release. Most commands in this book work the same on either, but paths differ where noted — e.g. `/etc/kernel/cmdline` (systemd-boot, typical on VE 9) versus `/etc/default/grub` (GRUB, common on older VE 8 and BIOS-mode installs) in Chapter 4. If you’re still on VE 8, plan the upgrade to VE 9 before investing time in passthrough setup.

Chapter 1

Overview

1.1 Target architecture

```
[ motherboard BIOS: VT-x/SVM + VT-d/AMD-Vi ]
      ↓
[ Proxmox kernel: intel/amd_iommu=on iommu=pt + vfio ]
      ↓
[ HBA bound to vfio-pci, blacklisted from host SAS driver ]
      ↓
[ TrueNAS VM: q35 + OVMF + hostpci0 ...,pcie=1,rombar=0 ]
      ↓
[ TrueNAS owns the HBA and the data pool ]
```

Proxmox boots from its own NVMe or board SATA. TrueNAS boots from a small virtual disk. The HBA and every disk on it belong only to the VM.

The same host can pass a second device — a discrete GPU — into another VM using the identical BIOS/IOMMU/vfio pipeline; see Chapter 9. The GPU and the HBA just need to land in separate IOMMU groups.

1.2 Why pass the HBA, not individual disks

Virtual disks and “passthrough by serial” sit behind the hypervisor. ZFS then cannot trust SMART, cache flush, or device identity the way it can on bare metal. Passing the **entire controller** gives TrueNAS the same view it would have on metal: serials, SMART, resilver, and disk replace.

Do **not** put the Proxmox boot drive on that HBA.

1.3 Hardware checklist

- CPU and board with IOMMU: Intel **VT-d** or AMD **AMD-Vi**
- Dedicated HBA flashed to **IT mode** (not IR/RAID)
- Cards that behave well: LSI/Broadcom 9207-8i, 9300-8i, 9400-8i, Dell H310 in IT mode
- Confirm firmware with `sas2flash -list` or `sas3flash -list` — look for (IT)
- RAM for the VM: 16 GB is a floor; more if the pool is large
- HBA in a PCIe slot that can land in its own IOMMU group (often slot 1 on consumer boards)
- If also passing a GPU: a board with an iGPU so the host keeps a console, or a plan to run headless (Chapter 9)

 Warning

IR/RAID firmware hides disks behind a RAID abstraction. ZFS needs raw devices. Flash IT mode **before** you pass the card through.

Chapter 2

BIOS and UEFI settings

Reboot the physical host and enter setup (Del, F2, or F10). Names vary by vendor. Enable every row that exists.

2.1 Settings that matter

Setting	Set to	Why
Intel VT-x or AMD SVM / AMD-V	Enabled	Basic virtualization. Not IOMMU.
Intel VT-d or AMD AMD-Vi / IOMMU	Enabled	Required for PCI passthrough.
SR-IOV	Enabled if present	Not required for an HBA.
Above 4G Decoding	Enabled	Required for large BAR allocations; more important when also passing a GPU, but harmless for an HBA-only build.

Setting	Set to	Why
Resizable BAR (Intel)	Enabled if also passing a GPU, otherwise off	Recommended alongside Above 4G Decoding for modern GPUs; leave off for an HBA-only build.
Resizable BAR (AMD)	Disabled	Known to cause boot/stability issues on several AMD platforms — leave off even when also passing a GPU.
CSM / Legacy boot	Off if Proxmox is UEFI	Stay in UEFI end to end.
Secure Boot	Off (simplest)	Avoids vfio module signing fights.
Fast Boot	Off	Fast Boot can skip device init steps the installer and IOMMU setup rely on.
PXE network boot	Off unless you need it	One less legacy boot path to fight with UEFI/OVMF.
iGPU / Integrated Graphics	Disable if the VM hangs after adding the HBA	Common Intel hang fix.
Slot Option ROM / OPROM for the HBA	Disabled if available	Stops the card from trying to boot the host.
Other Option ROMs (NIC, RAID controller, etc.)	Disabled if you see detection errors	Same failure mode as the HBA's Option ROM, just on a different slot.

Save and reboot into Proxmox.

i Note

Resizable BAR splits by vendor: Intel boards generally tolerate it and benefit when a GPU is passed through, while several AMD boards have reported crashes or failed boots with it enabled — regardless of whether a GPU is involved. When in doubt, leave it off and rely on Above 4G Decoding alone; only test enabling Resizable BAR on Intel systems after everything else is stable.

2.2 Where vendors hide IOMMU

- **Intel:** Advanced → CPU Configuration, System Agent, or Chipset. Labels: *VT-d*, *Intel VT for Directed I/O*, *IOMMU*.
- **AMD:** Advanced → AMD CBS → NBIO, or next to *SVM Mode*. Labels: *IOMMU*, *AMD-Vi*. On some boards SVM Mode instead lives under **OC (Overclocking)** → **CPU Features**.

! Important

VT-x / SVM alone is not enough. Without VT-d or AMD-Vi, `/sys/kernel/iommu_groups` stays empty and Proxmox will refuse the PCI device.

💡 Tip

Example: Gigabyte Z390 AORUS PRO WIFI-CF (Intel i9-9900).

On this board (AMI UEFI, BIOS F14a) the settings above live under Intel-labeled menus rather than the AMD ones — VT-x/VT-d under Advanced → CPU Configuration, Above 4G Decoding and Resizable BAR support under Advanced → Chipset. Being an Intel Z390/Coffee Lake platform, the Intel rows in the table apply (Resizable BAR only needed if also passing through a GPU); the AMD-specific rows and quirks (SVM path, forced-off Resizable BAR) don't apply here.

2.3 Slot choice

On many B450/B550/X570 boards the HBA only isolates cleanly in the **first full-length PCIe slot**. If Chapter 5 shows a shared group, move the card before you add ACS override.

Chapter 3

Installing Proxmox VE

With the BIOS settings from Chapter 2 in place, install Proxmox VE itself before moving on to IOMMU and passthrough configuration.

i Note

The HBA and GPU should already be seated in the system at this point — some boards renumber PCIe slots or change IOMMU grouping when cards are added or removed later. The drives on the HBA do **not** need to be connected yet, and no other storage needs to be attached; add them once you're ready to pass the card through to a VM.

3.1 Preparing installation media

- Get the current Proxmox VE installer ISO from the Proxmox downloads page.
- If reusing a drive that had a prior OS or partition table on it, wipe it first with **GParted** to avoid stale ESP or RAID metadata confusing the installer.
- Flash the ISO to a USB stick with **balenaEtcher** (or Rufus on Windows).

3.2 Booting the installer

- Plug in the USB stick, reboot, and open the boot menu (usually F11/F12/Esc — varies by vendor).
- Pick the boot entry explicitly labeled **UEFI: [your USB drive]**. Booting the non-UEFI/legacy entry produces a BIOS-mode install, which fights with the OVMF (UEFI) VMs used later in this book.

3.3 Running the installer

- Choose **Install Proxmox VE (Graphical)** and step through the EULA and target disk selection.
- For the target filesystem, ext4 is the simpler choice for the boot disk; ZFS is also offered by the installer but adds boot-pool complexity that isn't needed here. Either way the installer detects UEFI and creates the EFI System Partition (ESP) automatically.
- Set the host's network, timezone, and root password/email as prompted, then let it finish and reboot.

After the reboot, confirm the host actually came up in UEFI mode:

```
proxmox-boot-tool status
```

You want to see **System currently booted with uefi**. If it instead shows **bios/legacy**, the boot entry chosen during install was wrong — re-check the BIOS boot-mode settings from Chapter 2 and reinstall.

Tip

Additional data drives can be added to Proxmox at any time — they don't need to be present for this install. The HBA's drives specifically should stay disconnected until you're ready to pass the controller through to the TrueNAS VM in Chapter 8.

Chapter 4

Enable IOMMU on the Proxmox host

SSH into the host. See how it boots:

```
proxmox-boot-tool status
```

4.1 GRUB

Typical on BIOS installs and some older UEFI installs. Edit `/etc/default/grub`.

Intel:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=on iommu=pt"
```

AMD (modern kernels often enable AMD-Vi without the extra flag; keeping it is fine):

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet amd_iommu=on iommu=pt"
```

Apply:

```
update-grub
```

4.2 systemd-boot

Typical on a current UEFI Proxmox VE 9 install (also applies to VE 8, now end-of-life, if you haven't upgraded). Edit `/etc/kernel/cmdline` and **append** to the existing line. Do not delete `root=`.

Intel:

```
intel_iommu=on iommu=pt
```

AMD:

```
amd_iommu=on iommu=pt
```

Apply:

```
proxmox-boot-tool refresh
```

`iommu=pt` is passthrough mode: only devices given to VMs pay IOMMU cost.

i Note

On Proxmox kernels 6.8+, Intel often turns IOMMU on without `intel_iommu=on`. Keep the flag anyway so the intent is obvious in `/proc/cmdline`.

4.3 Load VFIO at boot

Add to `/etc/modules`:

```
vfio
vfio_iommu_type1
vfio_pci
```

`vfio_virqfd` was folded into other modules on newer kernels. Skip it if `modprobe` complains.

Reboot, then check:

```
dmesg | grep -e DMAR -e IOMMU -e AMD-Vi
cat /proc/cmdline
```

You want DMAR or AMD-Vi tables found, “IOMMU enabled”, and the flags in the command line.

Chapter 5

Identify the HBA and IOMMU group

5.1 Find the card

```
lspci -nn | grep -iE 'sas|scsi|lsi|broadcom|aha|hba'
```

Example output:

```
02:00.0 Serial Attached SCSI controller [0107]: Broadcom / LSI SAS3008 [1000:0097]
```

- 02:00.0 is the PCI address
- 1000:0097 is vendor:device (needed for vfio)

5.2 Print IOMMU groups

```
for d in /sys/kernel/iommu_groups/*/devices/*; do
  g=${d#/iommu_groups/}; g=${g%/*}
  echo "IOMMU Group $g: $(lspci -nns ${d##*/})"
done | sort -V
```

The HBA should sit **alone** in its group, or only with its own extra functions (02:00.0, 02:00.1, ...).

5.3 When the group is dirty

If the HBA shares a group with the management NIC, GPU, chipset USB, or the Proxmox boot controller, do not pass that group. The host will lose those devices the moment the VM starts.

Fix in this order:

1. Move the HBA to another PCIe slot (try slot 1).
2. Update the motherboard BIOS.
3. Last resort on a single-tenant homelab only: add to the kernel command line:

```
pcie_acs_override=downstream,multifunction
```

Then `update-grub` or `proxmox-boot-tool refresh` and reboot.

 Warning

ACS override weakens DMA isolation between devices. Fine on a box only you use. Do not use it on a shared host.

Chapter 6

Bind the HBA to vfio-pci

If Proxmox loads `mpt3sas`, `mpt2sas`, or `megaraid_sas` on the card, the host can see — and touch — the TrueNAS disks. Bind the card to `vfio` before you create the VM.

6.1 Confirm IDs

```
lspci -nn -s 02:00.0
```

Use **your** address and ID. The rest of this chapter keeps `02:00.0` and `1000:0097` as the example.

6.2 Bind and blacklist

```
echo "options vfio-pci ids=1000:0097" > /etc/modprobe.d/vfio.conf
echo "blacklist mpt3sas" >> /etc/modprobe.d/blacklist.conf
echo "blacklist mpt2sas" >> /etc/modprobe.d/blacklist.conf
update-initramfs -u -k all
reboot
```

After reboot:

```
lspci -k -s 02:00.0
```

Kernel driver in use: must be `vfio-pci`, not `mpt3sas`. The data disks should vanish from the Proxmox **Disks** view.

6.3 If the host driver still wins

```
echo "softdep mpt3sas pre: vfio-pci" >> /etc/modprobe.d/vfio.conf  
update-initramfs -u -k all  
reboot
```

Multi-function cards: put every vendor:device ID on one line, comma-separated:

```
options vfio-pci ids=1000:0097,1000:00c9
```

Chapter 7

Create the TrueNAS SCALE VM

Install TrueNAS onto a **virtual OS disk** first, with no HBA attached. Confirm it boots. Add passthrough in Chapter 8.

7.1 Download the TrueNAS SCALE ISO

- Get the ISO from the TrueNAS download page. As of this writing, **25.10 “Goldeye”** is the current stable line (25.10.7 at the time of writing) — pick that, not the TrueNAS 26 beta and not Nightlies, unless you specifically need a newer feature and accept the risk.
- Verify the download against the checksum published next to it on that page:

```
sha256sum TrueNAS-SCALE-*.iso
```

- Get the ISO onto Proxmox storage. Either upload it through the UI (**Datacenter** → **node** → **local (or other storage)** → **ISO Images** → **Upload**), or pull it straight from the Proxmox shell into the storage’s ISO directory:

```
wget -O /var/lib/vz/template/iso/TrueNAS-SCALE-latest.iso \
https://download.sys.truenas.net/TrueNAS-SCALE-Goldeye/latest/TrueNAS-SCALE.iso
```

Replace the URL with the one from the download page — the codename and version change with each release (it was “Dragonfish” for the 24.04 line; “Goldeye” is 25.10). **local** is the default storage name for ISOs; use whatever storage you configured for ISO Images if it differs.

7.2 Settings that matter

Setting	Value
Machine	q35
BIOS	OVMF (UEFI)
CPU type	host
Cores	4 or more
RAM	16 GB minimum, ballooning off
SCSI controller (OS disk only)	VirtIO SCSI
OS disk	16–32 GB on Proxmox SSD/NVMe
Network	VirtIO on vmbr0
Start at boot	Off until passthrough works

q35 is required for real PCIe. i440fx only does legacy PCI.

7.3 Example `qm create`

```
qm create 300 \
  --name truenas \
  --memory 16384 \
  --balloon 0 \
  --cores 4 \
  --cpu host \
  --bios ovmf \
  --machine q35 \
  --efidisk0 local-lvm:1,efitype=4m,pre-enrolled-keys=0 \
  --scsihw virtio-scsi-single \
  --scsi0 local-lvm:32,ssd=1 \
  --ide2 local:iso/TrueNAS-SCALE-latest.iso,media=cdrom \
  --boot order=ide2\;scsi0 \
  --net0 virtio,bridge=vmbr0 \
  --ostype l26
```

--ide2 ...,media=cdrom attaches the ISO downloaded above; swap in your actual filename. --boot order=ide2;scsi0 boots the installer first, then falls back to the virtual disk.

Start the VM, open its **Console** in the Proxmox UI, and run through the TrueNAS SCALE installer onto the virtual disk (**scsi0**) — the only disk it should see at this point, since the HBA isn't attached yet. Once TrueNAS boots on its own from **scsi0**:

```
qm set 300 --boot order=scsi0
qm set 300 --ide2 none
```

This drops the ISO and returns the boot order to the virtual disk only, so the VM doesn't try to reinstall on every reboot.

 Tip

Keep the TrueNAS boot pool on the virtual disk. The HBA is only for the data pool. You can rebuild the VM and import the same pool later.

Chapter 8

Pass the HBA into the VM

8.1 From the Proxmox UI

VM → **Hardware** → **Add** → **PCI Device**.

- Device: 02:00.0, or the whole 02:00 if you tick **All Functions**
- **PCI-Express**: on (pcie=1)
- **ROM-Bar**: off (rombar=0)
- Primary GPU: off

ROM-Bar off is the usual fix for a VM that hangs on “Booting from Hard Disk” after an LSI card is added.

8.2 From the CLI

One function:

```
qm set 300 --hostpci0 02:00.0,pcie=1,rombar=0
```

Every function at that address (preferred for HBAs):

```
qm set 300 --hostpci0 02:00,pcie=1,rombar=0
```

Do **not** make the HBA a boot device. TrueNAS boots from the virtual OS disk.

8.3 After start

In TrueNAS: **Storage** → **Disks**. You should see real model names, serials, and SMART. Import an existing pool or create a new one.

The matching line in `/etc/pve/qemu-server/300.conf` looks like:

```
hostpci0: 0000:02:00,pcie=1,rombar=0
```

i Note

Passing a GPU instead of an HBA? The mechanics are the same UI/CLI flow, but the flag values differ — see Chapter 9.

Chapter 9

GPU passthrough

A second common passthrough target on the same host is a discrete GPU — for a Windows gaming VM, a transcode/AI VM, or anything that needs direct hardware acceleration. The BIOS, IOMMU, and vfio groundwork from Chapter 2, Chapter 4, and Chapter 5 is identical; this chapter covers what’s different for a GPU versus the HBA in Chapter 6 and Chapter 8.

9.1 Identify the GPU and its group

```
lspci -nn | grep -iE 'vga|3d|display'
```

```
01:00.0 VGA compatible controller [0300]: NVIDIA Corporation ... [10de:2504]  
01:00.1 Audio device [0403]: NVIDIA Corporation ... [10de:228e]
```

A discrete GPU is almost always a **multi-function** device: the VGA function and its HDMI/DP audio function share one IOMMU group. Pass both together — check the group with the same loop from Chapter 5.

9.2 Free up a console for the host

If the discrete GPU is the only display adapter, the host needs another way to show a console once the card is handed to a VM:

- Prefer a board with an **iGPU** and set it as the primary/boot display in BIOS (see the iGPU row in Chapter 2), leaving the discrete card untouched at boot.
- Otherwise, plan to manage Proxmox entirely over SSH/the web UI after the card is bound to vfio.

9.3 Bind the GPU to vfio-pci

Same mechanism as the HBA, but blacklist the GPU driver instead of the SAS driver:

```
echo "options vfio-pci ids=10de:2504,10de:228e" > /etc/modprobe.d/vfio.conf
echo "blacklist nouveau" >> /etc/modprobe.d/blacklist.conf
echo "blacklist nvidia" >> /etc/modprobe.d/blacklist.conf
echo "blacklist amdgpu" >> /etc/modprobe.d/blacklist.conf
echo "blacklist radeon" >> /etc/modprobe.d/blacklist.conf
update-initramfs -u -k all
reboot
```

List both vendor:device IDs (VGA and audio function) on the `vfio-pci ids=` line. Only blacklist the driver family that matches your card — leave the other vendor's driver alone.

After reboot:

```
lspci -k -s 01:00.0
lspci -k -s 01:00.1
```

Both should show `Kernel driver in use: vfio-pci`. If the GPU driver still wins the race, add a `softdep` the same way as Chapter 6:

```
echo "softdep nvidia pre: vfio-pci" >> /etc/modprobe.d/vfio.conf
```

9.4 Pass the GPU into the VM

From the Proxmox UI: VM → **Hardware** → **Add** → **PCI Device**, tick **All Functions** so the VGA and audio functions move together.

From the CLI:

```
qm set 300 --hostpci0 01:00,pcie=1,x-vga=0
```

Notes on the flags, which differ from the HBA case in Chapter 8:

- **ROM-Bar:** leave **on** (default) for most GPUs so the guest can read the card's own VBIOS. Only set `rombar=0` with a `romfile=` (a VBIOS dump for that exact card) if the guest reports a black screen or Code 43 — this is common with some consumer Nvidia cards.
- **x-vga:** set `x-vga=1` only if this GPU is meant to be the VM's primary display shown in the Proxmox console framebuffer. For remote-desktop/headless use (RDP, Sunshine/Moonlight, transcoding), leave it 0.
- **Primary GPU** toggle in the UI mirrors `x-vga` — leave it off unless you need the boot-time framebuffer.

9.5 Known quirks

Warning

Nvidia “Code 43” in a Windows guest: Nvidia consumer drivers detect they’re in a VM and disable themselves. Hide the hypervisor from the guest:

```
qm set 300 --cpu host,hidden=1,flags=+pcid
```

Older Proxmox: add args: `-cpu host,kvm=off` to `/etc/pve/qemu-server/300.conf` instead.

Warning

AMD reset bug: some AMD cards don’t reset cleanly when the VM stops or reboots, leaving the GPU unusable until the host reboots. Look up whether your specific card/driver combination needs the `vendor-reset` kernel module before relying on VM restarts.

9.6 Verify

```
lspci -k -s 01:00.0
```

Then boot the VM and confirm the guest OS installs the real GPU driver and shows the card in Device Manager / `nvidia-smi` / `rocm-smi`, not a generic QXL/VGA adapter.

Chapter 10

TrueNAS after passthrough

10.1 First checks

- Disks appear with serial numbers, not generic **sdX** only
- SMART works inside TrueNAS, not on the Proxmox host
- Pool import or create succeeds without “disk in use” errors

10.2 Guest hygiene

- Static MAC on the VirtIO NIC, plus a DHCP reservation or static IP
- Do not use Proxmox snapshots of the virtual OS disk as the backup plan for the pool
- Snapshot and replicate **inside** TrueNAS (or **zfs send** to another box)
- VirtIO networking is enough for SMB/NFS. Pass a spare NIC later if you want 10 GbE with less CPU

10.3 What not to do

- Do not let Proxmox mount or SMART-scan the HBA disks
- Do not put swap or the Proxmox boot disk on the passed-through card
- Do not enable QEMU disk cache tricks on the data pool — there is no QEMU disk in front of it

Chapter 11

Troubleshooting

11.1 VM hangs at “Booting from Hard Disk”

Set `rombar=0` (uncheck **ROM-Bar**). Disable the slot Option ROM in system BIOS. Some LSI utilities also have “boot from this controller” — turn that off. The VM should boot the virtual OS disk only.

11.2 IOMMU groups missing / “PCI device not available”

VT-d or AMD-Vi is still off, or the kernel flags did not apply.

```
cat /proc/cmdline  
dmesg | grep -e DMAR -e IOMMU -e AMD-Vi
```

Revisit Chapter 2 and Chapter 4.

11.3 Proxmox still lists the data disks

`vfio` bind failed. `lspci -k -s 02:00.0` should show `vfio-pci`. Rebuild `initramfs` after blacklist changes (Chapter 6).

11.4 TrueNAS sees the HBA but no disks

- Confirm IT firmware, not IR
- Pass **all functions** of the card
- Reseat SFF-8087 / 8643 cables; the card can enumerate with no SAS link
- Check disk power

11.5 Host dies when the VM autostarts

The HBA shared an IOMMU group with the management NIC. From the boot menu, drop `intel_iommu=on` / `amd_iommu=on` for one boot, turn VM autostart off, fix grouping (Chapter 5), then re-enable IOMMU.

11.6 Need ACS override?

Only after a slot change and a BIOS update still leave a dirty group. Homelab only.

11.7 Guest shows a generic display adapter, not the GPU

The GPU driver still won on the host, so the guest is stuck with QXL/VGA emulation. Confirm `lspci -k -s <addr>` shows `vfio-pci` for **both** the VGA and audio functions (Chapter 9), and that both vendor:device IDs are on the `vfio-pci ids=` line.

11.8 Windows guest shows Nvidia Code 43

The driver detected virtualization. Hide it with `cpu: host,hidden=1` (Chapter 9).

11.9 GPU passthrough works once, then fails after a VM reboot

Likely the AMD reset bug — the card isn't releasing cleanly. Reboot the host, or look into `vendor-reset` for your card (Chapter 9).

11.10 Quick verification set

```
cat /proc/cmdline
dmesg | grep -e DMAR -e IOMMU -e vfio
lspci -k -s 02:00.0
ls /sys/kernel/iommu_groups
```

Chapter 12

Command reference

12.1 Host

```
proxmox-boot-tool status
cat /proc/cmdline
dmesg | grep -e DMAR -e IOMMU -e AMD-Vi -e vfio
lspci -nn | grep -iE 'sas|scsi|lsi|broadcom'
lspci -k -s 02:00.0
```

IOMMU groups:

```
for d in /sys/kernel/iommu_groups/*/devices/*; do
  g=${d#/iommu_groups/}; g=${g%/*}
  echo "IOMMU Group $g: $(lspci -nns ${d##*/})"
done | sort -V
```

12.2 Kernel flags

Intel GRUB:

```
quiet intel_iommu=on iommu=pt
```

AMD GRUB:

```
quiet amd_iommu=on iommu=pt
```

Optional last-resort ACS split:

```
pcie_acs_override=downstream,multifunction
```

12.3 vfio

```
# /etc/modprobe.d/vfio.conf
options vfio-pci ids=1000:0097
softdep mpt3sas pre: vfio-pci

# /etc/modprobe.d/blacklist.conf
blacklist mpt3sas
blacklist mpt2sas

update-initramfs -u -k all
```

12.4 VM

```
qm set 300 --hostpci0 02:00,pcie=1,rombar=0
```

/etc/pve/qemu-server/300.conf fragment:

```
bios: ovmf
machine: q35
cpu: host
balloon: 0
hostpci0: 0000:02:00,pcie=1,rombar=0
```

Replace 02:00 and 1000:0097 with your `lspci -nn` output.

12.5 GPU passthrough (Chapter 9)

```
lspci -nn | grep -iE 'vga|3d|display'
```

```
# /etc/modprobe.d/vfio.conf
options vfio-pci ids=10de:2504,10de:228e
```

```
# /etc/modprobe.d/blacklist.conf
blacklist nouveau
blacklist nvidia
```

```
qm set 300 --hostpci0 01:00,pcie=1,x-vga=0
qm set 300 --cpu host,hidden=1,flags=+pcid
```